Predictive Modeling For Breast Cancer

Ann Abigail Halim
Computer Science and Statistics
BINUS University
Jakarta, Indonesia
ann.halim@binus.ac.id

Hazel Zaki Adityo
Computer Science and Statistics
BINUS University
Jakarta, Indonesia
hazel.adityo@binus.ac.id

Joycelin
Computer Science and Statistics
BINUS University
Jakarta, Indonesia
joycelin@binus.ac.id

Marcell Prasetyo
Computer Science and Statistics
BINUS University
Jakarta, Indonesia
marcell.prasetyo@binus.ac.id

***Abstract*—Breast cancer remains one of the leading causes of mortality among women worldwide. Early diagnosis and accurate prognosis are critical challenges in cancer management. This study aims to explore the effectiveness of supervised machine learning models in predicting breast cancer survival outcomes using clinical and demographic data from the SEER-based Breast Cancer dataset. The dataset consists of 4,024 female patients diagnosed between 2006 and 2010, featuring variables such as age, tumor size, hormone receptor status, and lymph node involvement. Through a series of preprocessing steps, exploratory data analysis, and feature engineering, five predictive models, including Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and AdaBoost were developed and evaluated. Model performance was assessed using accuracy, precision, recall, and F1-score. Among the models, Random Forest achieved the highest predictive accuracy. The findings emphasize the potential of machine learning in supporting clinical decision-making, identifying high-risk patients, and improving overall survival strategies in breast cancer treatment.**

***Keywords*—**AdaBoost, Breast Cancer, Decision Tree, Logistic Regression, Machine Learning, Naïve Bayes, Predictive Modeling, Random Forest, Naive Bayes

I. INTRODUCTION

Breast cancer appears to be one of the most common and lethal diseases impacting women globally. The World Health Organization reports that breast cancer is the most frequently diagnosed cancer worldwide, with approximately 2.3 million new cases and 685,000 deaths in 2020 [1]. These figures indicate a variable yet generally ascending trend, underscoring the pressing necessity for enhanced early detection instruments and predictive analytics.

Conventional diagnostic techniques, including biopsies, mammograms, and clinical assessments, although efficacious, may be costly, time-intensive, and prone to variability stemming from human interpretation. This generates a necessity for precise, rapid, and scalable solutions to facilitate clinical decision-making, especially in resource-limited settings [1].

In recent years, machine learning (ML) has surfaced as a viable methodology to aid in medical diagnostics, particularly in breast cancer detection. Machine learning models can analyze historical data to discern intricate patterns and provide predictions with limited human involvement [2]. Logistic Regression, Decision Tree, and Random Forest are useful algorithms for binary classification issues, such as differentiating between benign and malignant tumors [3].

This work seeks to build and assess predictive models for breast cancer categorization utilizing these five algorithms for classifying the survival status of breast cancer patients based on clinical and demographic features and identify the most influential features contributing to breast cancer survival, thereby supporting clinical insights and decision-making. All models are developed in Python and trained using the Breast Cancer dataset sourced from Kaggle, contributed by Reihaneh Namdari and last updated in 2022 [4]. Before model training, statistical feature selection using the Chi-square test is employed to determine the most pertinent attributes [5]. Additionally, hyperparameter adjustment is performed via Random Search and Grid Search to enhance model performance [6].

II. RELATED WORK

A number of studies have explored the application of machine learning (ML) methodologies for breast cancer prediction, utilizing diverse classification algorithms and clinical datasets. One of the similar cases is the Wisconsin Breast Cancer Dataset (WBCD), which historically functioned as a standard due to its organized attributes and binary categorization labels [9]. Nonetheless, the emergence of publicly accessible medical data platforms like Kaggle has led researchers to increasingly employ real-world datasets that encompass a wider array of clinical and demographic characteristics. The Breast Cancer dataset by Reihaneh Namdari (2022) on Kaggle has facilitated more sophisticated modelling with the inclusion of survival status and further clinical variables [2].

Logistic Regression (LR), Support Vector Machines (SVM), Decision Trees (DT), Random Forests (RF), and Gradient Boosting Machines (GBM) are often utilized techniques due to their efficacy in binary classification applications. Patil et al. [10] examined SVM, Naïve Bayes, and Random Forest classifiers, concluding that ensemble approaches, such as Random Forest, surpass individual models in accuracy and resilience. Qiu et al. [11] highlighted

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

the significance of Decision Trees in terms of interpretability, while also acknowledging the potential for overfitting in the absence of pruning. Logistic Regression is favored for its simplicity and probabilistic interpretation, especially in clinical applications.

Feature selection is essential for improving the performance of prediction models. Researchers have employed methodologies including Recursive Feature Elimination (RFE), Principal Component Analysis (PCA), and statistical evaluations such as the Chi-square test to ascertain significant predictors. Chandrashekar and Sahin [12] showed the efficacy of Chi-square-based selection in diminishing dimensionality and enhancing model generalization. Hyperparameter tuning has emerged as a fundamental element of machine learning operations. Methods like Grid Search and Random Search, along with cross-validation, are commonly employed to optimize model parameters and mitigate overfitting [13], [14].

Regardless of these developments, a deficiency persists in research that amalgamates model comparison, statistical feature selection, and systematic hyperparameter tuning into a cohesive pipeline utilizing open-source tools. This study fills that gap by employing Python-based machine learning modules to develop and analyze five supervised models: Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and AdaBoost using the Breast Cancer dataset obtained from Kaggle. The objective is threefold: (1) to ascertain the most significant features affecting breast cancer survival through Chi-square analysis, (2) to develop predictive models for classifying survival outcomes based on clinical and demographic variables, and (3) to assess and compare model efficacy relying on metrics such as accuracy, precision, recall, and F1-score.

In addition to conventional models, Naive Bayes has been widely used as a baseline classifier due to its simplicity and strong probabilistic foundations. Despite its assumption of conditional independence, it has demonstrated reasonable performance in medical datasets. AdaBoost, as an ensemble boosting technique, has also gained attention in healthcare applications for its ability to improve classification accuracy by sequentially training weak learners. Both models offer complementary insights when compared with more complex ensemble and linear models.

III. OBJECTIVES

A. *To ascertain the most significant factors affecting breast cancer survival, thereby enhancing clinical understanding and decision-making.*

B. *To create prediction models employing supervised machine learning algorithms for the classification of breast cancer patients' survival status based on clinical and demographic characteristics.*

C. *To compare the performance of five supervised machine learning algorithms in predicting breast cancer patient survival, and identify the model with the highest predictive performance based on classification metrics.*

IV. METHODOLOGY

A. Data Collection

The dataset utilized in this study was sourced from Kaggle, specifically the Breast Cancer dataset from Reihaneh Namdari, last updated in 2022. It comprises 4,024 records with 16 variables, including patient demographics such as age, race, and marital status. The dataset additionally includes clinical characteristics such as tumor dimensions, cancer stage, hormone receptor status, and survival statistics. The dependent variable is Survival Status, denoting whether the patient is "Alive" or "Dead".

B. Data Preprocessing

1) Data Loading

The dataset was loaded into Jupyter Notebook with the pandas library's read_csv() function. The preliminary examination included verifying the absence of values with .info() and .isnull().sum(). Finally, verify the count of the target variable values using .value_counts() to ensure the data is not imbalanced.

2) Exploratory Data Analysis (EDA)

In exploratory data analysis (EDA), data is displayed with the matplotlib and seaborn libraries to create scatter plots and box plots. Seaborn's heatmap effectively facilitates the identification of correlations across variables, highlighting strongly connected aspects. Additionally, t-tests and chi-square tests were performed to ascertain statistically significant factors.

C. Model Building

1) Data Splitting

The dataset is partitioned into training and testing subsets, including 70% for training and 30% for testing. Utilizing the train_test_split() function from the sklearn library to guarantee a fair representation of surviving classes.

2) Model Development

This research employs five methodologies, which are logistic regression, decision tree, random forest, naïve bayes, and adaboost to create models and optimize accuracy. Hyperparameter adjustment is employed for each approach to achieve optimal accuracy.

D. Model Evaluation

The sklearn library was applied to compute precision, recall, and F1-score for evaluating the model's classification report. The confusion matrix was produced utilizing the confusion_matrix() and classification_report() functions to assess prediction results and model accuracy.

V. RESULT AND DISCUSSION

A. Dataset Description

The dataset used is original data from the Surveillance, Epidemiology, and End Result (SEER) Program of the National Cancer Institute (NCI). The data contains data on female breast cancer sufferers diagnosed with infiltrating duct and lobular carcinoma taken from the period 2006 to 2010.

The details of the data are as follows:

- Period : 2006 – 2010
- Gender : Female
- Amount : 4024 Patients

Criteria that are not included:

- Patients with unknown tumor size.
- Patients with an unknown number of regional lymph nodes were examined.
- Patients with an unknown number of positive regional lymph nodes.
- Patients with survival less than 1 month after diagnosis.

The number of patients obtained from the data is 4024. This dataset aims to support statistical analysis in survival analysis studies and evaluations in certain types of breast cancer patients.

B. Data Preprocessing

1) Importing Libraries

The dataset was initially loaded using Python libraries, particularly pandas, with preliminary inspections to check for data completeness and structural consistency. No missing value was detected. Outlier detection was conducted via boxplots; however, outliers were retained due to the clinical integrity of the dataset. Exploratory Data Analysis (EDA) using matplotlib and seaborn helps visualize distributions, detect skewness, and observe relationships among variables through scatter plots, boxplots, and heatmaps. Statistical significance was tested using chi-square tests for categorical variables and t-tests for numerical ones, with a significance threshold set at $\alpha = 0.05$.

2) Importing Dataset Information

Using the .info() function, the dataset information was shown as below.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4024 entries, 0 to 4023
Data columns (total 16 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Age                     4024 non-null   int64
 1   Race                    4024 non-null   object
 2   Marital Status          4024 non-null   object
 3   T Stage                 4024 non-null   object
 4   N Stage                 4024 non-null   object
 5   6th Stage               4024 non-null   object
 6   differentiate           4024 non-null   object
 7   Grade                   4024 non-null   object
 8   A Stage                 4024 non-null   object
 9   Tumor Size              4024 non-null   int64
 10  Estrogen Status         4024 non-null   object
 11  Progesterone Status     4024 non-null   object
 12  Regional Node Examined  4024 non-null   int64
 13  Reginol Node Positive   4024 non-null   int64
 14  Survival Months         4024 non-null   int64
 15  Status                  4024 non-null   object
dtypes: int64(5), object(11)
memory usage: 503.1+ KB
```

Fig 1. Dataset information summary showing non-null values and data types for each feature.

This output confirms that none of the variables contain missing values. Additionally, it displays the data types of each column, indicating whether the values are stored as integers or strings.

To evaluate the distribution of the target variable, the .value_counts() function was applied to the Status column.

Status	count
Alive	3408
Dead	616

Fig 2. Class label distribution within the target variable, "Status"

The output reveals that 3,408 instances are labeled as "Alive" and 616 as "Dead", indicating a significant class imbalance in the dataset.

3) Data Cleaning

The data cleaning stage is to handle data from incomplete, missing, and error values. Fortunately, there is no missing value (null) and our group has decided to accept outliers since the dataset comes from a medical standpoint. Therefore, there is no need to mutate the data.

4) Data Exploration

The data exploration stage is carried out to understand the characteristics of the dataset and ensure the suitability of the data for further statistical analysis. This is proven by one of the data visualizations, namely by creating graphs/charts. The goal is to find out whether the data to be used is appropriate and there are no obstacles such as missing values, input errors, and outlier data.

a) Status Class Distribution

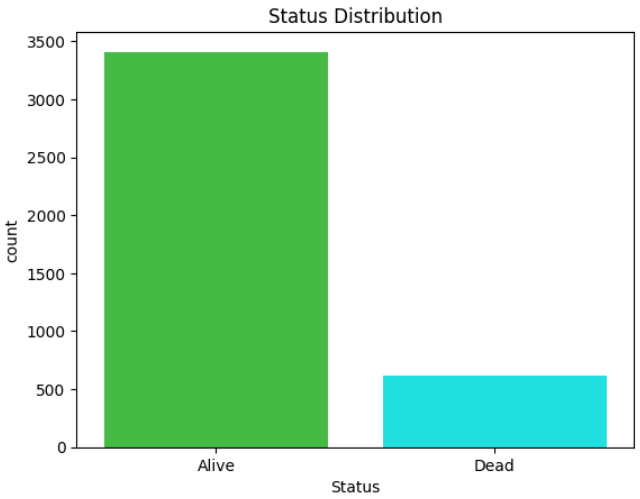


Fig 3. Distribution of survival status, indicating higher frequency of "Alive" class compared to "Dead."

The bar chart above shows the distribution of the values in the categorical variable "*Status*", with many more patients being alive than those who are dead, which causes an imbalance of data since this variable is the target.

b) Race Distribution

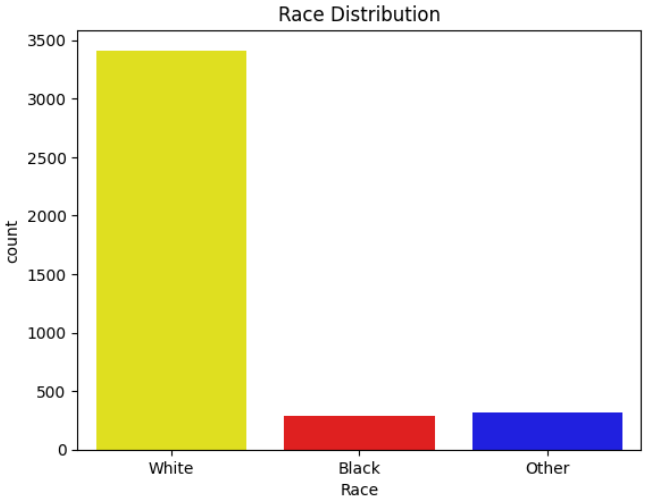


Fig 4. Race distribution among patients, showing the predominance of patients identified as White.

The bar chart above shows the distribution of the values in the categorical variable "*Race*", with many more patients identifying as white than those who identify from black or other races.

c) Numerical Variable Distribution

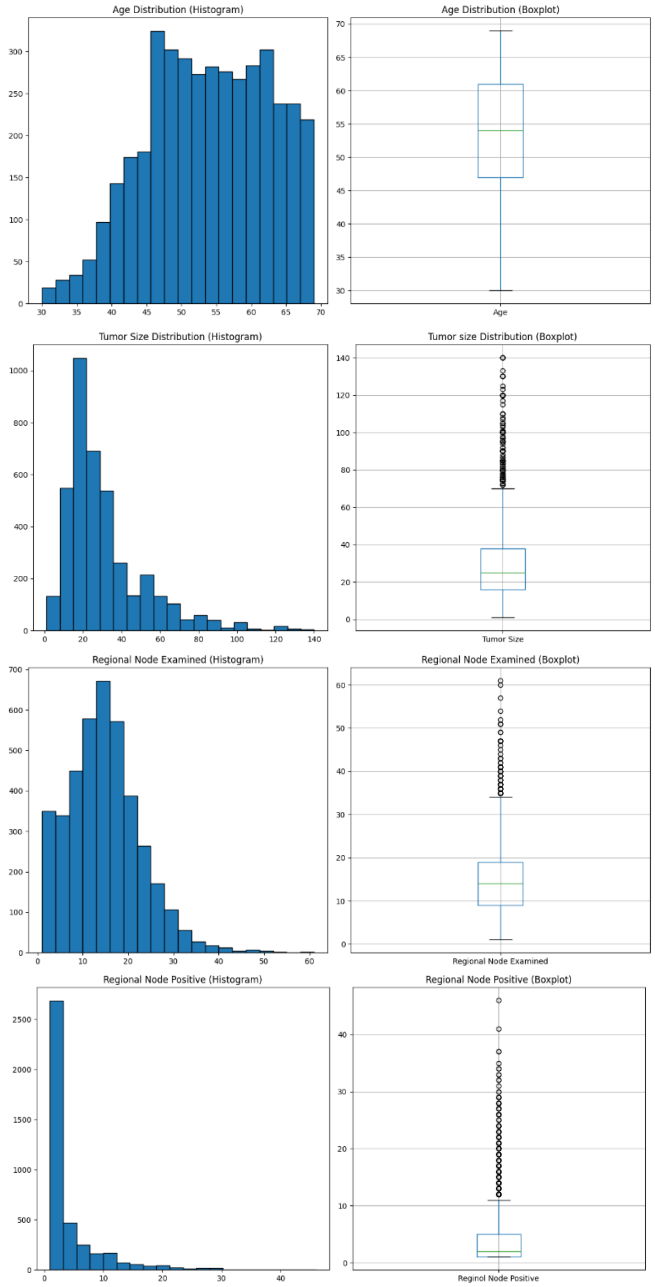


Fig 5-8. Histogram and boxplot visualizations of numerical features, illustrating data skewness and presence of outliers.

All the graph above shows the distribution of the values within the corresponding numerical variable column. The left graph is to show the distribution using a simple histogram plot with x-axis being the values within the corresponding variable grouped into intervals and the y-axis being the total count of values inside those intervals. On the other hand, the right graph is the boxplot, which is used to detect if any outliers are present within the corresponding variable and also to show the Five-Number Summary, which consists of the lowest value which is represented by the lowest point in the boxplot, first, second, and third quantile which is represented by the rectangle in the middle of the boxplot where 50% of the total data is present, and finally the highest value which is represented by the highest value in the boxplot.

d) Relationship between Age and Other Numerical Variables

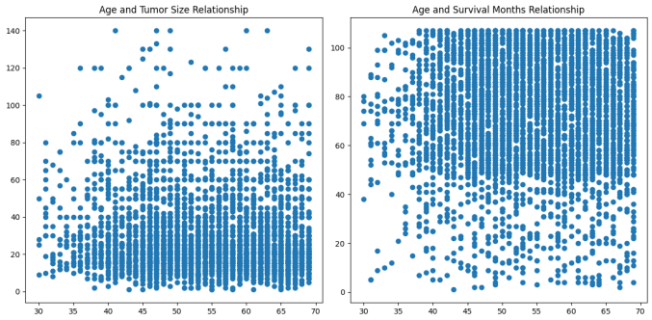


Fig 9. Scatter plots showing relationships between age and other numerical features (tumor size and survival months).

The scatterplot above is used to show the relationship between age and other numerical variables, and in this case those variables are tumor size and survival months on the y-axis and age on the x-axis. This is to show if there are any trends present in those relationships and from the graph above, we can conclude that there are no trends since no obvious linear or non-linear relationship is present.

e) Relationship between Age and the Status

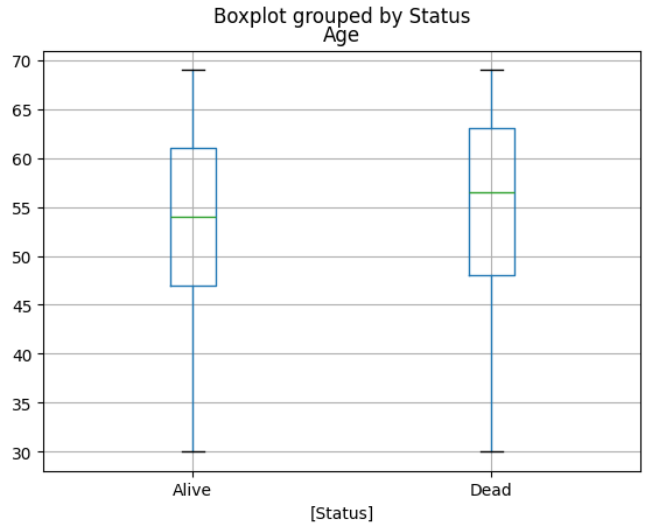


Fig 10. Boxplot comparison of age distribution across survival classes, indicating older average age among deceased patients.

The boxplot above is used to show the distribution of age within each class in the target variable, which is the status of the patients. From the graph above, we can see that 50% of the patients' age in the "dead" class is older than those in the "alive" class.

f) Correlation Analysis

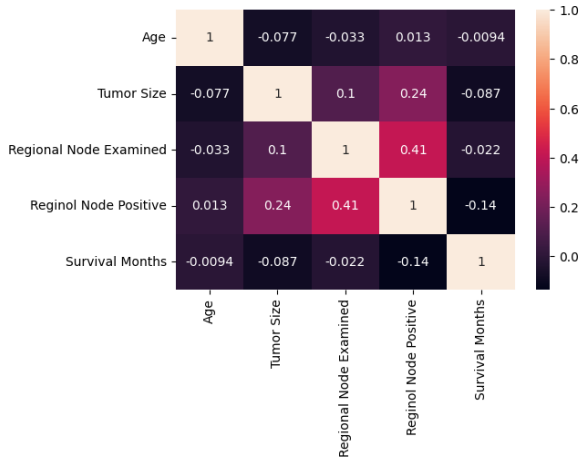


Fig 11. Heatmap of Pearson correlation among numerical variables, with strongest correlation observed between regional node counts.

The graph above is used to show the correlation between each numerical variables, to indicate the strength and direction of each relationship. From this we can conclude that only Regional Node Examined and Regional Node Positive have a somewhat strong positive relationship with each other.

g) Significance Test

In this part lies the significance test of variable in the dataset, with respect to the target variable. This is to show if the predictor variable within the dataset is significant in influencing the outcome class of the target variable. To do this, we used the *chi-square test* for the categorical variables and *t-test* for the numerical variables, where a variable is determined to be significant or not based on the comparison of its p-value and alpha value and in this case, we use the alpha value of 0.05 so if the corresponding variable's p-value is lower than 0.05, than it is considered to be significant.

Variable	Chi2	p-value	Significant
Race	27.970007	8.440929e-07	Yes
Marital Status	28.263813	1.102769e-05	Yes
T Stage	103.476309	2.779095e-22	Yes
N Stage	269.929143	2.430141e-59	Yes
6th Stage	281.648443	9.830332e-60	Yes
differentiate	112.556276	3.091352e-24	Yes
Grade	112.556276	3.091352e-24	Yes
A Stage	35.764727	2.226426e-09	Yes
Estrogen Status	135.155739	3.052608e-31	Yes
Progesterone Status	124.885388	5.392080e-29	Yes

Fig 12. Chi-square test results evaluating significance of categorical features with respect to survival status.

	Variable	T-statistic	p-value	Significant
0	Age	-3.3229	9.309078e-04	Yes
1	Tumor Size	-7.6265	7.019568e-14	Yes
2	Regional Node Examined	-2.1223	3.411297e-02	Yes
3	Reginol Node Positive	-11.9771	3.440363e-30	Yes
4	Survival Months	29.7029	1.192571e-129	Yes

Fig 13. T-tests results assessing the statistical significance of numerical variables in relation to survival outcome.

5) Feature Engineering

Categorical features were transformed for model compatibility. One-Hot Encoding was applied to non-ordinal attributes such as race, marital status, hormone receptor status, and stage classifications. Ordinal variables like tumor stage (T and N), grade, and differentiation were numerically mapped according to their clinical hierarchy. This ensured that all variables were in an appropriate format for modeling algorithms.

C. Model Building

a) Libraries

This study utilized various Python-based libraries for data preprocessing, model training, and evaluation. The primary library used for machine learning tasks was scikit-learn, which provided access to classification algorithms such as Logistic Regression, Decision Tree, Random Forest, Gaussian Naive Bayes, and AdaBoost. Hyperparameter tuning was implemented using RandomizedSearchCV and GridSearchCV, while performance metrics were derived using built-in evaluation functions including accuracy, precision, recall, and weighted F1-score.

To address class imbalance within the survival status variable, the Synthetic Minority Oversampling Technique (SMOTE) from the imblearn library was applied to the training set. Statistical distributions for parameter tuning, such as uniform and integer ranges, were imported from the scipy.stats module. Data manipulation and visualization were supported using libraries such as pandas, numpy, matplotlib, and seaborn.

b) Data Splitting

The data was split into training (70%) and testing (30%) sets using stratified sampling to maintain class proportions. Due to significant class imbalance between the “Alive” and “Dead” survival statuses, the SMOTE (Synthetic Minority Oversampling Technique) method was applied to the training set to synthesize new minority class samples and improve model generalization.

c) Labelling

```
x_train = df_train.drop('Status', axis = 1)
y_train = df_train['Status']
x_test = df_test.drop('Status', axis = 1)
y_test = df_test['Status']

x_train.columns = x_train.columns.astype(str)
x_test.columns = x_test.columns.astype(str)

from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
x_train_smote, y_train_smote =
smote.fit_resample(x_train, y_train)
```

The features and target variable (Status) were separated into independent (X) and dependent (y) variables for both training and testing datasets. Column names were standardized into string format to ensure compatibility with the modeling functions. Afterward, SMOTE was applied exclusively to the training data to balance the classes before training.

d) Building Models

Logistic Regression

Logistic Regression was trained with optimized hyperparameters using Randomized SearchCV. Parameters such as regularization strength (C), penalty type (l1, l2), and solver (liblinear, saga) were tuned. The model demonstrated significantly improved performance after tuning, achieving over 90% in weighted F1-score, which reflects its enhanced capability to distinguish between survival outcomes when properly regularized.

```
param_dist_log = {
    'C': stats.uniform(0.01, 10),
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga']
}

log_model_tuned =
LogisticRegression(max_iter=100)
random_search_log =
RandomizedSearchCV(log_model_tuned,
param_distributions=param_dist_log,
n_iter=100, cv=5, n_jobs=-1, random_state=42)

random_search_log.fit(x_train_smote,
y_train_smote)

best_log_random =
random_search_log.best_estimator_

best_log_random.fit(x_train, y_train)

y_test_predict_tuned_log =
best_log_random.predict(x_test)

cm_tuned_log = confusion_matrix(y_test,
y_test_predict_tuned_log)
result_tuned_log =
classification_report(y_test,
y_test_predict_tuned_log)
accuracy_tuned_log = accuracy_score(y_test,
y_test_predict_tuned_log)
f1_weight_log_tuned = f1_score(y_test,
y_test_predict_tuned_log, average='weighted')

print(f"Tuned Logistic Regression Accuracy:
{accuracy_tuned_log}")
print(cm_tuned_log)
print(result_tuned_log)
print("Weighted F1-score (Tuned Logistic
Regression):", f1_weight_log_tuned)
```

```
Accuracy: 0.9072847682119205
[[995  30]
 [ 82 101]]
              precision    recall  f1-score   support

       Alive       0.92      0.97      0.95      1025
        Dead       0.77      0.55      0.64       183

    accuracy                           0.91      1208
   macro avg       0.85      0.76      0.80      1208
weighted avg       0.90      0.91      0.90      1208

Weighted F1-score: 0.9007545214718364
```

Fig 14. Performance metrics of the tuned Logistic Regression model show a weighted F1-score of 90.07%.

From the output above, we could see the weighted f1-score is 0.9007, or around 90.07%.

Decision Tree

A decision tree classifier was developed with hyperparameter optimization using Randomized Search and 5-fold cross-validation. Parameters such as max_depth, min_samples_split, and min_samples_leaf were tuned to enhance model generalization and prevent overfitting. The optimized model was then evaluated on the test set.

```
param_dist = {
    'max_depth': randint(1, 20),
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
}

tree_reg = DecisionTreeClassifier()
random_search = RandomizedSearchCV(tree_reg,
param_distributions=param_dist, n_iter=100,
cv=5, n_jobs=-1, random_state=42)
random_search.fit(np.array(x_train_smote),
y_train_smote)
```

```
best_tree_random =
random_search.best_estimator_

best_tree_random.fit(x_train_smote,
y_train_smote)

y_test_predict =
best_tree_random.predict(x_test)
cm = confusion_matrix(y_test, y_test_predict)
result = classification_report(y_test,
y_test_predict)
accuracy = accuracy_score(y_test,
y_test_predict)
f1_weight_tree_random = f1_score(y_test,
y_test_predict, average='weighted')

print(f"Accuracy: {accuracy}")
print(cm)
print(result)
print("Weighted F1-score:",
f1_weight_tree_random
```

```
Accuracy: 0.8360927152317881
[[906 119]
 [ 79 104]]
              precision    recall  f1-score   support

       Alive       0.92      0.88      0.90      1025
        Dead       0.47      0.57      0.51       183

    accuracy                           0.84      1208
   macro avg       0.69      0.73      0.71      1208
weighted avg       0.85      0.84      0.84      1208

Weighted F1-score: 0.8425360474565806
```

Fig 15. Performance metrics of the tuned Decision Tree model yielded a weighted F1-score of 84.25%.

As the output above, the weighted f1-score of decision tree model is around 0.8425 or 84.25%, the lowest among the evaluated models.

Random Forest

A random forest classifier was also constructed with hyperparameter tuning via Randomized Search. Parameters including n_estimators, max_depth, and min_samples_split were optimized to improve classification performance. The final model demonstrated the strongest results among all models, showcasing its effectiveness in handling feature interactions and imbalanced data.

```
random.seed(123)

param_dist = {
    'n_estimators': randint(10, 100),
    'max_depth': randint(1, 20),
    'min_samples_split': randint(2, 11),

}

forest_reg = RandomForestClassifier()

random_search =
RandomizedSearchCV(forest_reg,
param_distributions=param_dist, n_iter=100,
cv=5, random_state=42, n_jobs=-1)
random_search.fit(np.array(x_train_smote),
y_train_smote)

best_random_model =
random_search.best_estimator_

best_random_model.fit(x_train, y_train)

y_test_predict =
best_random_model.predict(x_test)
cm = confusion_matrix(y_test, y_test_predict)
result = classification_report(y_test,
y_test_predict)
accuracy = accuracy_score(y_test,
y_test_predict)
f1_weight_forest_reg_random =
f1_score(y_test, y_test_predict,
average='weighted')

print(f"Accuracy: {accuracy}")
print(cm)
print(result)
```

```
print("Weighted F1-score:",
f1_weight_forest_reg_random)
```

```
Accuracy: 0.9172185430463576
[[1004   21]
 [  79  104]]
              precision    recall  f1-score   support

       Alive       0.93      0.98      0.95      1025
        Dead       0.83      0.57      0.68       183

    accuracy                           0.92      1208
   macro avg       0.88      0.77      0.81      1208
weighted avg       0.91      0.92      0.91      1208

Weighted F1-score: 0.9105630191716623
```

Fig 16. Performance metrics of the tuned Random Forest model, achieving the highest weighted F1-score of 91.05%.

This model achieved the highest performance, with the best weighted F1-score among all models, which is 0.9105 or 91.05% indicating its robustness and suitability for handling imbalanced, complex medical data.

NAÏVE BAYES

A Gaussian Naive Bayes classifier was also implemented. This model relies on the assumption of conditional independence between features and is computationally efficient. Although it yielded lower performance compared to ensemble methods, it provided a strong probabilistic baseline with reasonable predictive power and simplicity.

```
gnb = GaussianNB()

param_grid = {'var_smoothing':
np.logspace(0,-9, num=100)}

grid_search = GridSearchCV(estimator=gnb,
param_grid=param_grid, cv=5,
scoring='f1_weighted', n_jobs=-1)
grid_search.fit(x_train_smote, y_train_smote)

best_nb_model = grid_search.best_estimator_

y_test_predict =
best_nb_model.predict(x_test)

cm = confusion_matrix(y_test, y_test_predict)
result = classification_report(y_test,
y_test_predict)
accuracy = accuracy_score(y_test,
y_test_predict)
f1_weight_nb = f1_score(y_test,
y_test_predict, average='weighted')

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:")
print(cm)
print("Classification Report:")
print(result)
print("Weighted F1-score:", f1_weight_nb)
```

```
Accuracy: 0.8029801324503312
Confusion Matrix:
[[841 184]
 [ 54 129]]
Classification Report:
              precision    recall  f1-score   support

       Alive       0.94      0.82      0.88      1025
        Dead       0.41      0.70      0.52       183

    accuracy                           0.80      1208
   macro avg       0.68      0.76      0.70      1208
weighted avg       0.86      0.80      0.82      1208

Weighted F1-score: 0.8221293248860643
```

Fig 17. Performance metrics of the tuned Naive Bayes model, with a weighted F1-score of 82.21%

The result of weighted f1-score from the naïve bayes model is around 0.82.21 or 82.21%, which is the worse model compared with others.

ADABOOST

Finally, the AdaBoost (Adaptive Boosting) algorithm was applied to the training data. By combining multiple weak learners into a strong ensemble, AdaBoost effectively improved classification performance, particularly in handling class imbalance. The model was evaluated on the test set using standard metrics and demonstrated competitive performance among the tested models.

```
param_dist_ada = {
    'n_estimators': randint(50, 200),
    'learning_rate': stats.uniform(0.01,
1.0),
}

ada_model = AdaBoostClassifier()
random_search_ada =
RandomizedSearchCV(ada_model,
param_distributions=param_dist_ada,
n_iter=50, cv=5, n_jobs=-1, random_state=42)
# Reduced n_iter to 50
random_search_ada.fit(x_train_smote,
y_train_smote)

best_ada_random =
random_search_ada.best_estimator_

best_ada_random.fit(x_train_smote,
y_train_smote)

y_test_predict_ada =
best_ada_random.predict(x_test)
cm_ada = confusion_matrix(y_test,
y_test_predict_ada)
result_ada = classification_report(y_test,
y_test_predict_ada)
accuracy_ada = accuracy_score(y_test,
y_test_predict_ada)
f1_weight_ada_random = f1_score(y_test,
y_test_predict_ada, average='weighted')

print(f"Accuracy: {accuracy_ada}")
print(cm_ada)
print(result_ada)
```

```
print("Weighted F1-score:",
f1_weight_ada_random)
```

```
Accuracy: 0.875
[[929  96]
 [ 55 128]]
              precision    recall  f1-score   support

       Alive       0.94      0.91      0.92      1025
        Dead       0.57      0.70      0.63       183

    accuracy                           0.88      1208
   macro avg       0.76      0.80      0.78      1208
weighted avg       0.89      0.88      0.88      1208

Weighted F1-score: 0.8800205585919871
```

Fig 18. Performance metrics of the tuned AdaBoost model, showing a weighted F1-score of approximately 88%

The weighted f1-score of this model is around 0.88 or 88%, which is between decision tree and logistic regression.

D. Model Comparison

1) Weighted F1-score Comparison

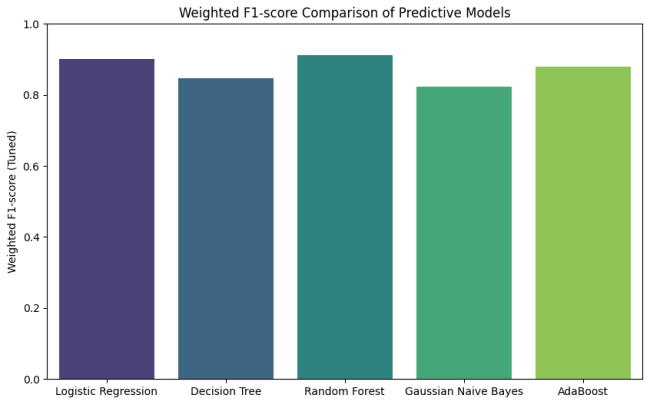


Fig 19. Comparative chart of weighted F1-scores across all five models, with Random Forest performing best, followed by Logistic Regression and AdaBoost.

The graph above illustrates the comparative performance of all five models based on their weighted F1-scores. The Random Forest model demonstrated the highest score at 91.05%, followed closely by Logistic Regression (90.07%) and AdaBoost (88%). Decision Tree achieved a score of 84.25%, while Naive Bayes, although simpler, provided a respectable 82.21%. These results indicate that ensemble models and optimized linear classifiers outperform simpler statistical models when applied to imbalanced, high-dimensional medical data. Weighted F1-score was selected to account for class imbalance, offering a single metric that balances precision and recall across both classes.

2) ROC Curve and AUC Evaluation

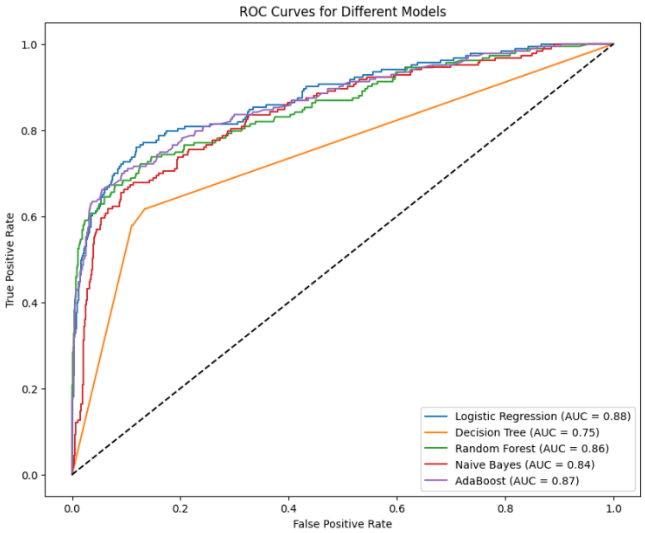


Fig. 20 ROC curves and AUC values for all five models. Logistic Regression achieved the highest AUC (0.88), followed by AdaBoost (0.87), Random Forest (0.86), Naive Bayes (0.84), and Decision Tree (0.75).

To further evaluate the discriminative power of each model, Receiver Operating Characteristic (ROC) curves were generated and the Area Under the Curve (AUC) scores were computed.

As shown in Fig. 20, Logistic Regression achieved the highest AUC score of 0.88, indicating a strong ability to distinguish between survival classes. AdaBoost followed with an AUC of 0.87, and Random Forest with 0.86. Naive Bayes also performed reasonably well with an AUC of 0.84. In contrast, Decision Tree obtained the lowest AUC score of 0.75, suggesting limited generalization despite high training performance.

These results align with the F1-score rankings and reinforce the conclusion that ensemble methods and tuned linear models outperform simpler classifiers in imbalanced clinical prediction tasks.

VI. CONCLUSION

A. Summary

The dataset used for this project is the Breast Cancer Dataset by Reihaneh Namdari, comprising 4,024 records of female patients diagnosed with invasive ductal and lobular carcinoma between 2006 and 2010. Sourced from the SEER program, the dataset includes clinical and demographic features such as age, race, marital status, tumor size, hormone receptor status, lymph node involvement, and survival status (Alive or Dead). The goal was to develop machine learning models to predict patient survival outcomes and identify key features influencing prognosis.

The data was preprocessed using Python, with steps including handling of categorical variables via one-hot encoding, outlier detection through boxplots, and analysis of feature distributions. Exploratory Data Analysis (EDA) showed that variables such as tumor size, lymph node positivity, and age had right-skewed distributions and meaningful correlations with survival. Count plots and scatterplots revealed imbalances in the dataset and patterns

across different groups, including survival distributions across age and race.

Feature engineering was performed to convert categorical features into numerical formats suitable for modeling. Following this, five supervised machine learning models including Logistic Regression, Decision Tree, Random Forest, Naïve Bayes, and AdaBoost were built to classify survival status. Performance was evaluated using accuracy, precision, recall, and F1-score. Among all models, Random Forest demonstrated superior performance in terms of classification metrics, reflecting its robustness in handling non-linear relationships and feature interactions.

Visualizations such as heatmaps confirmed the strongest correlations between tumor-related metrics and survival status. Moreover, returning patients with favorable hormone receptor status showed higher chances of survival, aligning with clinical expectations. Overall, the pipeline effectively demonstrated the role of machine learning in breast cancer survival prediction.

B. Conclusion

This study successfully applied supervised machine learning techniques to predict the survival status of breast cancer patients using real clinical data. Key insights and takeaways are outlined below:

- Random Forest consistently outperformed Logistic Regression, Decision Tree, Naïve Bayes, and AdaBoost models, suggesting that ensemble learning techniques offer more reliable classification performance on complex medical datasets.
- Tumor size, lymph node involvement, and age emerged as strong predictors of survival, reinforcing their clinical relevance in breast cancer prognosis and supporting the model's interpretability.
- Hormone receptor status (estrogen and progesterone) played a vital role in classification accuracy, highlighting their importance for targeted treatments and personalized medicine approaches.
- Modeling survival as a binary classification problem provides actionable outputs for clinicians, enabling early identification of high-risk patients and more effective treatment planning.

By implementing these data-driven strategies, healthcare providers can improve clinical decision-making, optimize patient outcomes, and support preventive efforts through predictive analytics. The results of this study emphasize the promising role of machine learning in oncology, and the importance of using comprehensive datasets to build intelligent, supportive diagnostic systems.

This study has several limitations. The dataset is limited to patients from a specific time period (2006–2010) and location (SEER program, USA), which may not generalize globally. In addition, the model treats survival as a binary classification problem rather than a time-to-event problem, limiting its prognostic depth. Future work may explore the application of survival analysis methods such as Cox Proportional Hazards models or deep learning-based survival models. Furthermore, incorporating model interpretability frameworks such as SHAP or LIME could enhance clinical transparency.

REFERENCES

[1] World Health Organization, "Breast cancer: key facts," WHO, 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/breast-cancer.

[2] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," in *Proceedings of the IEEE International Conference on Data Mining*, 2010, pp. 785–794.

[3] M. R. Ganggayah, N. Taib, S. Haron, and K. Hassan, "Predicting breast cancer survivability: a comparison of three data mining methods," in *Proc. 2018 Int. Conf. on Computer and Communication Engineering (ICCCE)*, Kuala Lumpur, Malaysia, 2018, pp. 210–214.

[4] R. Namdari, "Breast Cancer Dataset," Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/reihanehnamdari/breast-cancer-dataset.

[5] J. Zhang, X. Wang, and H. Liu, "Feature selection based on chi-square test," in *Proc. 2017 Int. Conf. on Computer Science and Information Technology*, 2017, pp. 18–22.

[6] L. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," in *Proc. 2012 Int. Conf. on Machine Learning*, 2012, pp. 281–288.

[7] W. Cahyana and N. Nurlayli, "Breast cancer detection using logistic regression," in *Proc. 2023 Int. Conf. on Biomedical Informatics*, 2023, pp. 56–60.

[8] M. R. Ganggayah, N. Taib, S. Haron, and K. Hassan, "Predicting breast cancer survivability: a comparison of three data mining methods," in *Proc. 2018 Int. Conf. on Computer and Communication Engineering (ICCCE)*, 2018, pp. 210–214.

[9] W. H. Wolberg, O. L. Mangasarian, "Multisurface method of pattern separation for medical diagnosis applied to breast cytology," Proc. Nat. Acad. Sci. USA, vol. 87, no. 23, pp. 9193–9196, 1990.

[10] V. M. Patil, N. L. Bhale, and A. V. Deorankar, "Breast cancer detection using machine learning techniques," in Proc. Int. Conf. Computing, Communication, Control and Automation (ICCUBEA), Pune, India, 2019, pp. 1–5.

[11] Y. Qiu, M. Lyu, and X. Shi, "A decision tree approach to predict breast cancer based on clinical data," in Proc. IEEE Int. Conf. Bioinformatics and Biomedicine (BIBM), Seoul, South Korea, 2020, pp. 1129–1135.

[12] G. Chandrashekar and F. Sahin, "A survey on feature selection methods," Computers & Electrical Engineering, vol. 40, no. 1, pp. 16–28, Jan. 2014.

[13] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281–305, Feb. 2012.

[14] P. Probst, M. Wright, and A. Boulesteix, "Hyperparameters and tuning strategies for random forest," Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, vol. 9, no. 3, p. e1301, 2018.